ResNet y Transfer Learning

Aprendizaje residual para redes extremadamente profundas, y cómo reutilizar lo aprendido
Basado en el artículo original (He et al., 2016) y en la Lecture 9 de CS231n (Stanford, 2017)

Universidad Nacional de Ingeniería — FIIS, Unidad de Posgrado
Redes Neuronales y Aprendizaje Profundo (Clase 6)

Sobre este documento

La sección de ResNet resume *Deep Residual Learning for Image Recognition* (K. He, X. Zhang, S. Ren, J. Sun, CVPR 2016; arXiv:1512.03385), complementado con la presentación que hacen Fei-Fei Li, Justin Johnson y Serena Yeung en la Lecture 9 de CS231n (Stanford, primavera 2017) sobre arquitecturas CNN — de allí provienen, en particular, el ejemplo numérico del costo del módulo Inception y la tabla comparativa de eficiencia (Canziani et al., 2017) citada por esa clase. La sección de *transfer learning* sintetiza la práctica estándar descrita en esa misma fuente y en la literatura asociada (Yosinski et al., 2014; Razavian et al., 2014). No se reproduce texto textual de ninguna fuente; todas las cifras provienen de ellas y se citan.

Índice

1. Contexto: la "revolución de la profundidad" **2**

2. El problema de la degradación **2**

3. Aprendizaje residual **3**

3.1. La idea central . . . 3
3.2. Atajos de identidad vs. de proyección . . . 4
3.3. El bloque *bottleneck* . . . 4
3.4. Arquitecturas completas: de 18 a 152 capas . . . 5
3.5. Entrenamiento . . . 5
3.6. Resultados en ImageNet . . . 6
3.7. Más allá de la clasificación: generalización . . . 7
3.8. Eficiencia comparada: el panorama completo . . . 7

4. Transfer Learning **7**

4.1. Motivación . . . 7
4.2. Dos estrategias principales . . . 8
4.3. ¿Cuál estrategia usar? Tamaño del dataset × similitud con la tarea original . . . 8
4.4. Detalles prácticos de implementación . . . 9

4.5. Ejemplo mínimo (PyTorch) 9
4.6. Por qué esto importa para el resto del curso 9

5. Resumen y preguntas de repaso **9**

1. Contexto: la "revolución de la profundidad"

Entre 2012 y 2015 el error top-5 en ILSVRC cayó de forma sostenida al aumentar la profundidad de las redes ganadoras:

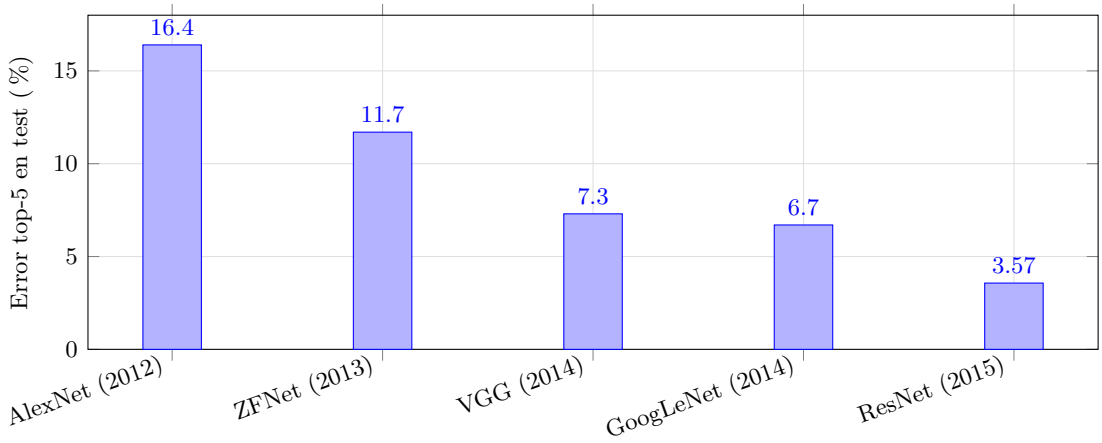


Figura 1. Error top-5 de los ganadores de ILSVRC 2012–2015. Datos: Simonyan & Zisserman (2015), Szegedy et al. (2015) y He et al. (2016, Tabla 5); ver también CS231n Lecture 9, diapositivas 22–25 y 64.

VGG (16–19 capas) y GoogLeNet (22 capas) ya habían llevado la profundidad muy lejos frente a AlexNet (8 capas). La pregunta natural, con la que abre el artículo de ResNet y con la que CS231n introduce el tema, es: *¿qué pasa si simplemente seguimos apilando más capas sobre una red convolucional "plana"?* La respuesta resulta ser más sutil de lo esperado.

2. El problema de la degradación

Antes de ResNet, se asumía que redes más profundas eran más difíciles de entrenar principalmente por el **desvanecimiento/explosión del gradiente**. Ese problema ya estaba en gran medida resuelto hacia 2015 mediante inicializaciones cuidadosas y *Batch Normalization* (Ioffe & Szegedy, 2015), que permiten que redes de decenas de capas empiecen a converger con SGD.

Sin embargo, al resolver ese problema apareció uno nuevo, inesperado: el **problema de la degradación**. Al comparar una red "plana" (simple pila de capas, al estilo VGG) de 20 capas contra una de 56 capas en CIFAR-10, la red de 56 capas tiene **mayor** error tanto de entrenamiento como de prueba que la de 20 capas. Esto **no es sobreajuste**: si lo fuera, el error de *entrenamiento* de la red más profunda sería menor (o igual), no mayor.

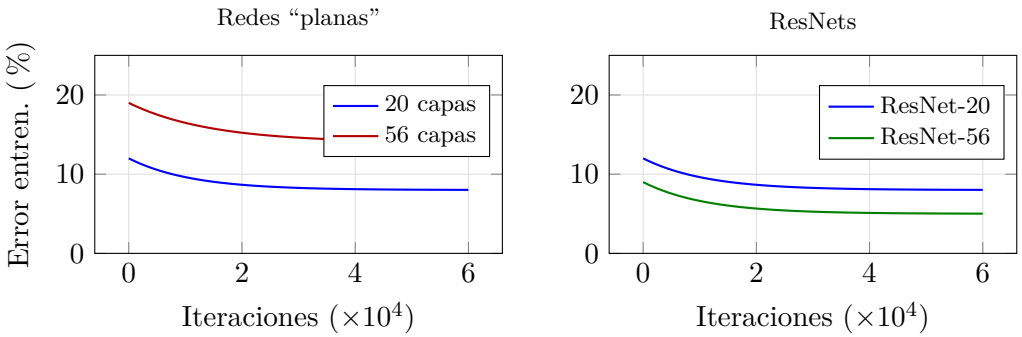


Figura 2. Reproducción esquemática de la Figura 1 y la Figura 6 del artículo (también diapositivas 67–68 de CS231n): una red plana de 56 capas entrena *peor* que una de 20 capas, mientras que la versión residual de 56 capas entrena mejor que la de 20. Curvas ilustrativas, no datos exactos del paper.

Observación clave (tanto del artículo como de CS231n): existe una solución trivial *por construcción* para que una red más profunda nunca sea peor que una más superficial: copiar las capas de la red superficial y hacer que las capas adicionales sean funciones identidad. El hecho de que esta solución exista, pero los optimizadores actuales no logren encontrarla (o algo igual de bueno) en tiempo razonable, sugiere que el problema es de **optimización** — las redes más profundas son más difíciles de optimizar — no de capacidad del modelo ni de sobreajuste.

3. Aprendizaje residual

3.1. La idea central

En vez de esperar que unas pocas capas apiladas aprendan directamente el mapeo deseado $H(x)$, se les hace aprender el **residuo**:

$$F(x) := H(x) - x \qquad \Longrightarrow \qquad H(x) = F(x) + x. \tag{1}$$

La hipótesis es que optimizar el mapeo residual $F(x)$ es más fácil que optimizar el mapeo original $H(x)$ sin referencia. En el caso extremo en que la identidad fuera óptima, es más fácil llevar $F(x)$ a cero que ajustar una identidad mediante una pila de capas no lineales.

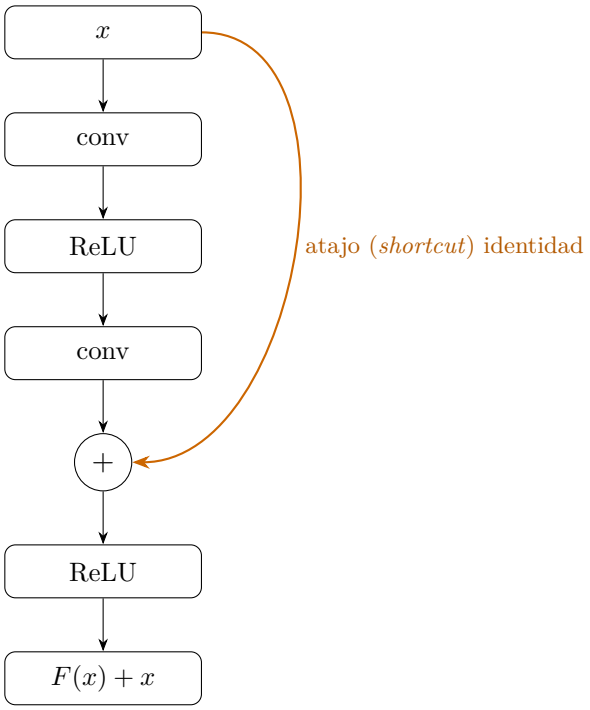


Figura 3. Bloque residual básico (Figura 2 del artículo; diapositivas 65–72 de CS231n): la conexión de atajo suma x a la salida de dos capas convolucionales, antes de la segunda ReLU.

Formalmente, para un bloque con dos capas: $y = F(x, \{W_i\}) + x$, con $F = W_2\,\sigma(W_1 x)$ (σ = ReLU, sesgos omitidos). Un punto crucial, señalado en ambas fuentes: **la conexión de atajo identidad no añade parámetros ni cómputo adicional** (salvo la suma elemento a elemento), lo que permite comparar de forma justa una red plana y su contraparte residual con el mismo número de parámetros, profundidad, ancho y costo computacional.

3.2. Atajos de identidad vs. de proyección

Si las dimensiones de x y $F(x)$ no coinciden (por ejemplo, al cambiar el número de canales entre etapas), se necesita una proyección lineal W_s en el atajo:

$$y = F(x, \{W_i\}) + W_s x. \tag{2}$$

El artículo compara tres opciones para las conexiones que cruzan etapas de distinta dimensión:

- **(A)** Atajo identidad con relleno de ceros para igualar dimensiones — sin parámetros extra.
- **(B)** Atajo de proyección (1×1 conv.) sólo donde cambian las dimensiones; el resto, identidad.
- **(C)** Todas las conexiones son proyecciones.

Los tres funcionan considerablemente mejor que la red plana; B es ligeramente mejor que A, y C apenas mejor que B (atribuible a los parámetros extra de las 13 proyecciones). Dado que las diferencias son pequeñas, el artículo concluye que las proyecciones **no son esenciales** para resolver la degradación, y usa la opción B como estándar (proyección sólo al cambiar dimensión) para no aumentar innecesariamente memoria y tiempo de cómputo.

3.3. El bloque *bottleneck*

Para las redes más profundas (50, 101 y 152 capas), usar bloques de 2 capas por razones de tiempo de entrenamiento sería costoso. En su lugar se usa un bloque de **3 capas**, con la misma lógica de

reducción de dimensión que las convoluciones 1×1 de GoogLeNet (CS231n lo señala explícitamente, diapositiva 78: "similar to GoogLeNet"): conv. 1×1 (reduce canales) $\rightarrow$ conv. 3×3 (opera en dimensión reducida) $\rightarrow$ conv. 1×1 (restaura canales).

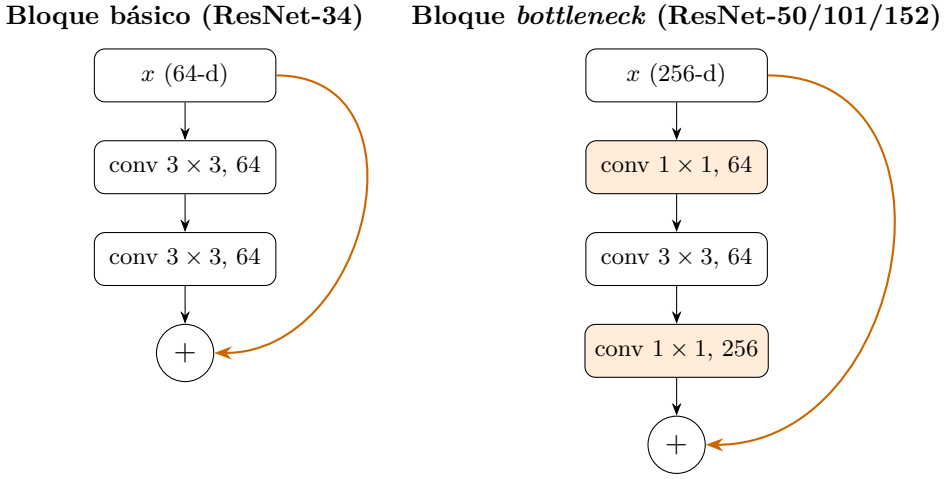


Figura 4. Bloque básico de 2 capas (izquierda, usado en ResNet-18/34) vs. bloque *bottleneck* de 3 capas (derecha, usado en ResNet-50/101/152). Ambos tienen complejidad temporal similar. Recreación de la Figura 5 del artículo y de la diapositiva 78–79 de CS231n.

Aquí el atajo identidad (sin parámetros) es especialmente importante: si se reemplazara por una proyección en el bloque *bottleneck*, el tamaño del modelo y el costo computacional se **duplicarían**, porque el atajo conectaría los dos extremos de alta dimensión (256-d) en vez de los de baja dimensión (64-d).

3.4. Arquitecturas completas: de 18 a 152 capas

Las reglas de diseño siguen la filosofía de VGG: todas las convoluciones son 3×3 (excepto los 1×1 del *bottleneck*); para el mismo tamaño de mapa de características, el número de filtros se mantiene igual; y periódicamente, cuando el tamaño del mapa se reduce a la mitad (stride 2), el número de filtros se **duplica** (diapositiva 74 de CS231n). Hay una capa convolucional adicional al inicio (7×7, stride 2) y, al final, *no hay capas FC ocultas*: sólo *global average pooling* seguido de una única capa FC-1000 — igual que en GoogLeNet (diapositiva 76).

Dato notable del artículo

El ResNet-34 tiene 3.6 mil millones de FLOPs — sólo el **18 %** de los 19.6 mil millones de VGG-19 — a pesar de tener una arquitectura comparable en espíritu (todo 3×3). Incluso el ResNet-152, con 152 capas, tiene **menos** complejidad (11.3 mil millones de FLOPs) que VGG-16/19 (15.3/19.6 mil millones).

3.5. Entrenamiento

- Aumento de datos: lado menor reescalado aleatoriamente en $[256, 480]$; recorte aleatorio 224×224 (o su espejo), con aumento de color estándar (como en AlexNet).
- **Batch Normalization** inmediatamente después de cada convolución y antes de la activación (Ioffe & Szegedy, 2015); sin dropout.
- Inicialización de pesos según He et al. (2015) — CS231n la resume como "Xavier/2"—, diseñada específicamente para capas con ReLU.

Cuadro 1: Arquitecturas de ResNet para ImageNet (Tabla 1 del artículo; diapositiva 77 de CS231n). Cada corchete es un bloque residual.

Capa	salida	18 capas	34 capas	50 capas	101/152 capas
conv1	112×112	7×7, 64, stride 2			
conv2_x	56×56	maxpool 3×3, stride 2			
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 23/36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$
		average pool 1×1, FC-1000, softmax			
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	$7.6/11.3 \times 10^9$

- SGD, tamaño de mini-lote 256, momento 0.9.
- Tasa de aprendizaje inicial 0.1, dividida por 10 cuando el error de validación se estanca.
- **Weight decay:** el artículo original reporta 10^{-4}; CS231n (diapositiva 80) resume la receta práctica con 10^{-5} — una pequeña discrepancia entre fuentes que vale la pena notar en clase.
- En test: *10-crop testing* estándar para comparaciones, o forma totalmente convolucional con promediado multi-escala (lado menor en $\{224, 256, 384, 480, 640\}$) para los mejores resultados.

3.6. Resultados en ImageNet

Red	Plana (error top-1, %)	ResNet (error top-1, %)
18 capas	27.94	27.88
34 capas	28.54	**25.03**

Con 18 capas, plana y residual rinden similar (el problema de degradación aún no se manifiesta), pero la versión residual **converge más rápido**. Con 34 capas, la red plana *empeora* respecto a la de 18 capas, mientras que la ResNet-34 *mejora* un 2.8 % sobre ResNet-18 — la degradación queda resuelta y la profundidad vuelve a ayudar.

Modelo	Error top-1 (%)	Error top-5 (%)
VGG-16	28.07	9.33
GoogLeNet	—	9.15
ResNet-34	24.19–25.03	7.40–7.76
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

El *ensemble* final de 6 modelos (dos de ellos ResNet-152) alcanza **3.57 %** de error top-5 en el conjunto de prueba de ImageNet, ganando el ILSVRC-2015. Tanto el artículo como CS231n destacan

que este resultado superó por primera vez el "desempeño humano" estimado por Russakovsky et al. (2014) en esta tarea.

3.7. Más allá de la clasificación: generalización

El mismo backbone, usado dentro de Faster R-CNN (Ren et al., 2015) en lugar de VGG-16, dio mejoras sustanciales **sin cambiar nada más que la red**:

	VGG-16	ResNet-101
mAP, PASCAL VOC 2007 test	73.2	**76.4**
mAP, PASCAL VOC 2012 test	70.4	**73.8**
mAP@[.5,.95], COCO val	21.2	**27.2** (+28 % relativo)

Con estas mejoras, los equipos basados en ResNet ganaron las 4 competencias principales de ILSVRC & COCO 2015: clasificación, detección y localización en ImageNet, y detección en COCO — lo que CS231n resume como "*swept all classification and detection competitions*".

3.8. Eficiencia comparada: el panorama completo

CS231n cierra el repaso de arquitecturas con las cifras de Canziani et al. (2017, *An Analysis of Deep Neural Network Models for Practical Applications*), que comparan exactitud, número de operaciones y consumo de memoria de todas las arquitecturas vistas:

Red	Rasgo distintivo (según CS231n)
AlexNet	Cómputo pequeño, pero aún pesado en memoria y menor exactitud
VGG	Mayor uso de memoria y de operaciones de toda la comparación
GoogLeNet	La más eficiente en cómputo por unidad de exactitud
ResNet	Eficiencia moderada (según el modelo), pero la **mayor exactitud**

Es decir: no existe una única arquitectura "mejor" en todos los ejes — la elección depende de si se optimiza por exactitud, por memoria o por velocidad de inferencia, algo especialmente relevante al desplegar modelos en producción o en dispositivos con recursos limitados.

Un detalle práctico importante para más adelante

Para usar ResNet como extractor de características en detección de objetos, el artículo calcula las estadísticas de Batch Normalization sobre el conjunto de entrenamiento de ImageNet **una sola vez**, tras el preentrenamiento, y luego **congela** esas capas BN durante el ajuste fino en la nueva tarea (se comportan como una transformación afín fija). Este es un ejemplo concreto — y temprano — de una práctica central en *transfer learning* con redes profundas, tema de la siguiente sección.

4. Transfer Learning

4.1. Motivación

Entrenar una red profunda desde cero, con inicialización aleatoria, requiere datasets del tamaño de ImageNet (1.2 millones de imágenes) y varias semanas de cómputo en GPU, como reportan tanto

el artículo de VGG como el de GoogLeNet. En la práctica, muy pocas personas entrenan una red convolucional completa desde cero: es más común **preentrenar** en un dataset grande (típicamente ImageNet) y luego reutilizar esa red para la tarea de interés, que casi siempre cuenta con muchos menos datos etiquetados.

La justificación conceptual: las primeras capas de una CNN aprenden características genéricas (bordes, texturas, manchas de color) que son útiles para prácticamente cualquier tarea de visión, mientras que las capas más profundas aprenden características cada vez más específicas de las clases del dataset original. De hecho, ya en la clase de VGGNet vimos que los propios autores señalan (y CS231n lo repite en la diapositiva 35) que las características de la capa FC7 (la penúltima) **generalizan bien a otras tareas**, incluso usadas con un clasificador lineal simple sin ajuste fino.

4.2. Dos estrategias principales

1. **Extractor de características fijo (*feature extraction*).** Se toma la red preentrenada, se le quita la(s) última(s) capa(s) de clasificación, y se **congelan** todos los demás pesos (no se calculan gradientes para ellos). Se entrena únicamente un clasificador nuevo (a menudo una sola capa lineal o FC) sobre las características extraídas.
2. **Ajuste fino (*fine-tuning*).** Además de reemplazar y entrenar el clasificador final, se continúan actualizando (algunos o todos) los pesos de la red preentrenada mediante retropropagación, normalmente con una **tasa de aprendizaje mucho menor** que la usada para entrenar desde cero, para no destruir las características útiles ya aprendidas.

4.3. ¿Cuál estrategia usar? Tamaño del dataset × similitud con la tarea original

La heurística estándar (presentada en CS231n y ampliamente usada en la práctica) organiza la decisión en una matriz de dos factores: qué tan **grande** es el nuevo dataset, y qué tan **similar** es a ImageNet (o al dataset de preentrenamiento).

	Dataset pequeño	**Dataset grande**
Similar a la tarea original	Riesgo de sobreajuste al ajustar capas profundas. **Recomendación:** extractor de características fijo — entrenar sólo un clasificador lineal sobre las características de las últimas capas.	Hay datos suficientes para adaptar la red sin sobreajustar. **Recomendación:** ajuste fino de toda (o casi toda) la red, con tasa de aprendizaje baja.
Diferente de la tarea original	El caso más difícil: pocos datos y las características de alto nivel (específicas de ImageNet) pueden no ser útiles. **Recomendación:** extractor de características fijo, pero tomando la salida de capas *más tempranas* (más genéricas) en vez de las últimas; o probar un clasificador más simple para evitar sobreajuste.	Suficientes datos para reaprender características específicas de la nueva tarea. **Recomendación:** ajuste fino de toda la red, partiendo de los pesos preentrenados como inicialización (mejor que aleatoria) y con más cuidado (tasa de aprendizaje baja al inicio, posible descongelamiento gradual de capas).

Cuadro 2: Matriz de decisión estándar para transfer learning (síntesis de CS231n y la literatura asociada).

4.4. Detalles prácticos de implementación

- **Tasa de aprendizaje.** Al hacer ajuste fino, se usa una tasa de aprendizaje considerablemente menor (por ejemplo, 10× a 100× menor) que la usada para entrenar la red original desde cero, ya que se espera que los pesos preentrenados estén relativamente cerca de un buen óptimo para la nueva tarea.
- **Descongelamiento gradual.** Una práctica común: primero entrenar sólo el clasificador nuevo con el resto de la red congelada (para que el clasificador "se estabilice"); luego descongelar progresivamente capas más profundas, de atrás hacia adelante, y seguir el ajuste fino con tasas de aprendizaje aún más bajas.
- **Congelar estadísticas de Batch Normalization.** Como hicieron He et al. al adaptar ResNet-101 para detección (ver nota de la sección anterior), suele ser conveniente fijar las medias/varianzas de BN calculadas durante el preentrenamiento, especialmente si los mini-lotes del ajuste fino son pequeños (las estadísticas de BN calculadas sobre lotes muy pequeños son ruidosas y pueden degradar el entrenamiento).
- **Elección del backbone.** Redes con *global average pooling* antes de la capa final (GoogLeNet, ResNet) son especialmente convenientes para transfer learning: basta con reemplazar la única capa FC final por una del tamaño adecuado al número de clases nuevo, sin preocuparse por recalcular las dimensiones de varias capas FC intermedias (como sí sería necesario en una VGG con FC de 7×7×512).

4.5. Ejemplo mínimo (PyTorch)

```
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Estrategia 1: extractor de características fijo
for param in model.parameters():
   param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, num_clases_nuevas)
   # sólo esta capa se entrena

# Estrategia 2: ajuste fino completo, con LR baja
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4, momentum=0.9)
```

Figura 5. Las dos estrategias de transfer learning en PyTorch: congelar todo excepto la capa final (extractor fijo), o permitir gradientes en toda la red pero con una tasa de aprendizaje mucho menor (ajuste fino).

4.6. Por qué esto importa para el resto del curso

La mayoría de las aplicaciones prácticas de visión por computadora — clasificación médica, inspección industrial, clasificación de especies, etc. — no cuentan con millones de imágenes etiquetadas. El patrón dominante en la industria y en la investigación aplicada es: tomar un backbone preentrenado en ImageNet (con frecuencia, una variante de ResNet, dada su combinación de exactitud y eficiencia razonable), y aplicar alguna de las estrategias de esta sección. Este patrón reaparecerá cuando veamos detección de objetos, segmentación y modelos de visión más recientes.

5. Resumen y preguntas de repaso

Ideas clave.

1. El problema de la degradación (no el desvanecimiento del gradiente) es lo que impedía entrenar redes planas extremadamente profundas; se manifiesta como mayor error de *entrenamiento*, no sólo de validación.
2. La reformulación residual $H(x) = F(x) + x$, mediante conexiones de atajo identidad, hace que aprender una identidad (o algo cercano) sea trivial, resolviendo la degradación sin añadir parámetros.
3. Los atajos de proyección son útiles sólo al cambiar dimensiones; en general, la identidad basta.
4. El bloque *bottleneck* ($1 \times 1 \to 3 \times 3 \to 1 \times 1$) permite escalar a 50–152 capas sin explotar el costo computacional, igual que la reducción de dimensión en GoogLeNet.
5. ResNet generaliza excepcionalmente bien como *backbone*: el mismo cambio de red (VGG-16 $\to$ ResNet-101) mejora sustancialmente detección y segmentación sin ningún otro cambio.
6. No existe una arquitectura óptima en todos los ejes: GoogLeNet es la más eficiente en cómputo, VGG la más pesada, y ResNet ofrece la mayor exactitud con eficiencia moderada (Canziani et al., 2017).
7. Transfer learning — extractor fijo o ajuste fino, según tamaño y similitud del dataset — es la forma estándar en que estas arquitecturas se usan en la práctica, más que entrenarlas desde cero.

Preguntas de repaso.

1. Explique por qué el problema de la degradación demuestra que la dificultad de entrenar redes muy profundas es de *optimización* y no de *capacidad* del modelo.
2. Si F tuviera una sola capa, la ecuación $y = F(x, \{W_i\}) + x$ se reduciría a una capa lineal más un atajo. ¿Por qué el artículo señala que no se observan ventajas en ese caso?
3. ¿Por qué el atajo identidad (sin parámetros) es más importante para el bloque *bottleneck* que para el bloque básico de 2 capas? (Piense en qué dimensiones conectaría una proyección en cada caso.)
4. Calcule el número de bloques residuales en cada etapa (conv2_x a conv5_x) para ResNet-101 usando la Tabla 1, y verifique que sumando 1 (conv1) + 3×(número de bloques) + 1 (FC) da 101 capas con pesos.
5. Un colega tiene 500 imágenes médicas etiquetadas en 3 clases, muy distintas visualmente de ImageNet. Según la matriz de decisión de la Sección 5.3, ¿qué estrategia de transfer learning recomendaría, y con qué capas trabajaría?
6. ¿Por qué es recomendable usar una tasa de aprendizaje menor al hacer ajuste fino que al entrenar desde cero?

Referencias

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*. arXiv:1512.03385.

Li, F.-F., Johnson, J., & Yeung, S. (2017). *CS231n: Convolutional Neural Networks for Visual Recognition*, Lecture 9: CNN Architectures. Stanford University.

Canziani, A., Paszke, A., & Culurciello, E. (2017). An Analysis of Deep Neural Network Models for Practical Applications. arXiv:1605.07678.

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *ICCV*.

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS*.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How Transferable Are Features in Deep Neural Networks? *NeurIPS*.

Razavian, A. S., Azizpour, H., Sullivan, J., & Carlsson, S. (2014). CNN Features Off-the-Shelf: An Astounding Baseline for Recognition. *CVPR Workshops*.

Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*.